

Decisiones de diseño

Los cambios realizados al programa fueron realizados para poder cumplir con los objetivos del proyecto, se colocaron los patrones en un orden específico dentro del MASTER.

¿Por qué?

El motor de regex evalúa de izquierda a derecha.
El primer patrón que coincide es el que se usa.

Ejemplo clave:

KEYWORD antes que IDENTIFIER

>>> antes que >>

comentarios antes que operadores

¿Qué problema resuelve? Evita ambigüedades.

El whitespace no se convierte en token, pero sí se procesa.

¿Por qué?

Porque es necesario para:

Llevar control de línea

Llevar control de columna

¿Cómo funciona?

Se recorre carácter por carácter:

for (char c : lexeme.toCharArray())

Si es \n:

aumenta línea

reinicia columna

Si no:

aumenta columna

Impacto

Permite ubicar tokens con precisión

Fundamental para mensajes de error

Inclusión de columna en los tokens

¿Qué se decidió?

Agregar el atributo column en el Token.

¿Por qué?

Antes solo se tenía la línea, lo cual es insuficiente para:

Ubicar errores con precisión

Debugging detallado

¿Cómo funciona?

Se inicializa en 1

Se incrementa según el tamaño del lexema

Se reinicia en cada salto de línea

Impacto

Salida más profesional

Similar a compiladores reales

8. Ignorar comentarios normales pero conservar JavaDoc

¿Qué se decidió?

Ignorar:

```
// comentario
```

```
/* comentario */
```

Conservar:

```
/** JavaDoc */
```

¿Por qué?

Los comentarios normales no afectan la ejecución, pero:

JavaDoc puede ser útil para documentación o análisis posterior

Impacto

Se optimiza el análisis

Se conserva información relevante

Resultados de pruebas

Caso 1

```
[L1 ] KEYWORD    → "public"
[L1 ] KEYWORD    → "class"
[L1 ] IDENTIFIER → "MiClase"
[L2 ] KEYWORD    → "private"
[L2 ] KEYWORD    → "int"
[L2 ] IDENTIFIER → "contador"
[L3 ] KEYWORD    → "public"
[L3 ] KEYWORD    → "void"
[L3 ] IDENTIFIER → "incrementar"
```

Caso 2

```
IDENTIFIER → "a"
OPERATOR   → "=="
IDENTIFIER → "b"
OPERATOR   → "&&"
IDENTIFIER → "c"
OPERATOR   → "!="
IDENTIFIER → "d"
OPERATOR   → "||"
IDENTIFIER → "i"
OPERATOR   → "++"
OPERATOR   → "<="
INTEGER_LIT → "10"
```

Caso 3

INTEGERLIT → "255"
HEXADECIMALLIT → "0xFF"
BINARYLIT → "0b1111_0000"
FLOATLIT → "3.14159"
STRINGLIT → "\"hola\\n\""

Caso 4

[L2] JAVADOC → "/* JavaDoc comment */"
[L3] KEYWORD → "public"
[L3] KEYWORD → "class"
[L3] IDENTIFIER → "Prueba"
[L4] KEYWORD → "public"
[L4] KEYWORD → "static"
[L4] KEYWORD → "void"
[L4] IDENTIFIER → "main"
[L5] HEX_LIT → "0xFF"
[L6] FLOAT_LIT → "3.14"
[L7] STRING_LIT → "\"mundo\""
[L8] OPERATOR → "=="
[L8] OPERATOR → "&&"
[L8] OPERATOR → ">"
[L8] OPERATOR → "++"
△ Error léxico [L9, C9]: '@' (U+0040)

Proyecto

LINE	COL	TYPE	LEXEME
------	-----	------	--------

analisislexico.Analisislexico\$LexicalError: Error léxico [L 125, C9]: '@' (U+0040)
' (U+000D)

analisislexico.Analisislexico\$LexicalError: Error léxico [L 125, C0]: '
' (U+000A)
' (U+000D)

analisislexico.Analisislexico\$LexicalError: Error léxico [L 125, C0]: '
' (U+000A)

[L2 C1] KEYWORD → "package"
[L2 C9] IDENTIFIER → "analisislexico"
[L2 C23] SEPARATOR → ";"
[L4 C1] JAVADOC → "/*
*
* @author namis
*/"

[L5 C1] KEYWORD → "public"
[L5 C8] KEYWORD → "class"
[L5 C14] IDENTIFIER → "Ejemplo"
[L5 C21] SEPARATOR → "{"
[L6 C5] JAVADOC → "/* Programa de prueba para el analizador léxico */"

[L7 C5]	KEYWORD	→ "public"
[L7 C12]	KEYWORD	→ "static"
[L7 C19]	KEYWORD	→ "void"
[L7 C24]	IDENTIFIER	→ "main"
[L7 C28]	SEPARATOR	→ "("
[L7 C29]	IDENTIFIER	→ "String"
[L7 C35]	SEPARATOR	→ "["
[L7 C36]	SEPARATOR	→ "]"
[L7 C38]	IDENTIFIER	→ "args"
[L7 C42]	SEPARATOR	→ ")"
[L7 C44]	SEPARATOR	→ "{"
[L9 C9]	KEYWORD	→ "int"
[L9 C13]	IDENTIFIER	→ "a"
[L9 C15]	OPERATOR	→ "="
[L9 C17]	INTEGERLIT	→ "10"
[L9 C19]	SEPARATOR	→ ";"
[L10 C9]	KEYWORD	→ "int"
[L10 C13]	IDENTIFIER	→ "b"
[L10 C15]	OPERATOR	→ "="
[L10 C17]	INTEGERLIT	→ "20"
[L10 C19]	SEPARATOR	→ ";"
[L11 C9]	KEYWORD	→ "int"
[L11 C13]	IDENTIFIER	→ "c"
[L11 C15]	OPERATOR	→ "="
[L11 C17]	IDENTIFIER	→ "a"
[L11 C19]	OPERATOR	→ "+"
[L11 C21]	IDENTIFIER	→ "b"
[L11 C22]	SEPARATOR	→ ";"
[L13 C9]	KEYWORD	→ "double"
[L13 C16]	IDENTIFIER	→ "pi"
[L13 C19]	OPERATOR	→ "="
[L13 C21]	FLOATLIT	→ "3.1416"
[L13 C27]	SEPARATOR	→ ";"
[L14 C9]	KEYWORD	→ "float"
[L14 C15]	IDENTIFIER	→ "f"
[L14 C17]	OPERATOR	→ "="
[L14 C19]	FLOATLIT	→ "2.5f"
[L14 C23]	SEPARATOR	→ ";"
[L16 C9]	IDENTIFIER	→ "String"
[L16 C16]	IDENTIFIER	→ "mensaje"
[L16 C24]	OPERATOR	→ "="
[L16 C26]	STRINGLIT	→ ""Hola mundo""
[L16 C38]	SEPARATOR	→ ";"
[L17 C9]	KEYWORD	→ "char"
[L17 C14]	IDENTIFIER	→ "letra"
[L17 C20]	OPERATOR	→ "="
[L17 C22]	CHARLIT	→ "'A'"
[L17 C25]	SEPARATOR	→ ";"

[L19 C9]	KEYWORD	→ "if"
[L19 C12]	SEPARATOR	→ "("
[L19 C13]	IDENTIFIER	→ "a"
[L19 C15]	OPERATOR	→ "<"
[L19 C17]	IDENTIFIER	→ "b"
[L19 C19]	OPERATOR	→ "&&"
[L19 C22]	IDENTIFIER	→ "b"
[L19 C24]	OPERATOR	→ ">"
[L19 C26]	INTEGERLIT	→ "0"
[L19 C27]	SEPARATOR	→ ")"
[L19 C29]	SEPARATOR	→ "{"
[L20 C13]	IDENTIFIER	→ "a"
[L20 C14]	OPERATOR	→ "++"
[L20 C16]	SEPARATOR	→ ";
[L21 C13]	IDENTIFIER	→ "b"
[L21 C14]	OPERATOR	→ "--"
[L21 C16]	SEPARATOR	→ ";
[L22 C9]	SEPARATOR	→ "}"
[L24 C9]	KEYWORD	→ "for"
[L24 C13]	SEPARATOR	→ "("
[L24 C14]	KEYWORD	→ "int"
[L24 C18]	IDENTIFIER	→ "i"
[L24 C20]	OPERATOR	→ "=="
[L24 C22]	INTEGERLIT	→ "0"
[L24 C23]	SEPARATOR	→ ";
[L24 C25]	IDENTIFIER	→ "i"
[L24 C27]	OPERATOR	→ "<"
[L24 C29]	INTEGERLIT	→ "10"
[L24 C31]	SEPARATOR	→ ";
[L24 C33]	IDENTIFIER	→ "i"
[L24 C34]	OPERATOR	→ "++"
[L24 C36]	SEPARATOR	→ ")"
[L24 C38]	SEPARATOR	→ "{"
[L25 C13]	IDENTIFIER	→ "System"
[L25 C19]	SEPARATOR	→ "."
[L25 C20]	IDENTIFIER	→ "out"
[L25 C23]	SEPARATOR	→ "."
[L25 C24]	IDENTIFIER	→ "println"
[L25 C31]	SEPARATOR	→ "("
[L25 C32]	STRINGLIT	→ ""Iteración: ""
[L25 C46]	OPERATOR	→ "+"
[L25 C48]	IDENTIFIER	→ "i"
[L25 C49]	SEPARATOR	→ ")"
[L25 C50]	SEPARATOR	→ ";
[L26 C9]	SEPARATOR	→ "}"
[L28 C9]	KEYWORD	→ "while"
[L28 C15]	SEPARATOR	→ "("
[L28 C16]	IDENTIFIER	→ "a"

[L28 C18] OPERATOR → ">"
 [L28 C20] INTEGERLIT → "0"
 [L28 C21] SEPARATOR → ")"
 [L28 C23] SEPARATOR → "{"
 [L29 C13] IDENTIFIER → "a"
 [L29 C14] OPERATOR → "--"
 [L29 C16] SEPARATOR → ","
 [L30 C9] SEPARATOR → "}"
 [L33 C9] KEYWORD → "int"
 [L33 C13] IDENTIFIER → "suma"
 [L33 C18] OPERATOR → "="
 [L33 C20] INTEGERLIT → "0"
 [L33 C21] SEPARATOR → ","
 [L34 C9] IDENTIFIER → "suma"
 [L34 C14] OPERATOR → "+="
 [L34 C17] INTEGERLIT → "1"
 [L34 C18] SEPARATOR → ","
 [L35 C9] IDENTIFIER → "suma"
 [L35 C14] OPERATOR → "+="
 [L35 C17] INTEGERLIT → "2"
 [L35 C18] SEPARATOR → ","
 [L36 C9] IDENTIFIER → "suma"
 [L36 C14] OPERATOR → "+="
 [L36 C17] INTEGERLIT → "3"
 [L36 C18] SEPARATOR → ","
 [L37 C9] IDENTIFIER → "suma"
 [L37 C14] OPERATOR → "+="
 [L37 C17] INTEGERLIT → "4"
 [L37 C18] SEPARATOR → ","
 [L38 C9] IDENTIFIER → "suma"
 [L38 C14] OPERATOR → "+="
 [L38 C17] INTEGERLIT → "5"
 [L38 C18] SEPARATOR → ","
 [L39 C9] IDENTIFIER → "suma"
 [L39 C14] OPERATOR → "+="
 [L39 C17] INTEGERLIT → "6"
 [L39 C18] SEPARATOR → ","
 [L40 C9] IDENTIFIER → "suma"
 [L40 C14] OPERATOR → "+="
 [L40 C17] INTEGERLIT → "7"
 [L40 C18] SEPARATOR → ","
 [L41 C9] IDENTIFIER → "suma"
 [L41 C14] OPERATOR → "+="
 [L41 C17] INTEGERLIT → "8"
 [L41 C18] SEPARATOR → ","
 [L42 C9] IDENTIFIER → "suma"
 [L42 C14] OPERATOR → "+="
 [L42 C17] INTEGERLIT → "9"

[L42 C18] SEPARATOR → ","
 [L43 C9] IDENTIFIER → "suma"
 [L43 C14] OPERATOR → "+="
 [L43 C17] INTEGERLIT → "10"
 [L43 C19] SEPARATOR → ","
 [L46 C9] KEYWORD → "int"
 [L46 C13] IDENTIFIER → "hex"
 [L46 C17] OPERATOR → "="
 [L46 C19] HEXADECIMALLIT → "0xFF"
 [L46 C23] SEPARATOR → ","
 [L47 C9] KEYWORD → "int"
 [L47 C13] IDENTIFIER → "bin"
 [L47 C17] OPERATOR → "="
 [L47 C19] BINARYLIT → "0b1010"
 [L47 C25] SEPARATOR → ","
 [L48 C9] KEYWORD → "int"
 [L48 C13] IDENTIFIER → "oct"
 [L48 C17] OPERATOR → "="
 [L48 C19] OCTALLIT → "077"
 [L48 C22] SEPARATOR → ","
 [L50 C9] KEYWORD → "double"
 [L50 C16] IDENTIFIER → "num1"
 [L50 C21] OPERATOR → "="
 [L50 C23] FLOATLIT → "1.1"
 [L50 C26] SEPARATOR → ","
 [L51 C9] KEYWORD → "double"
 [L51 C16] IDENTIFIER → "num2"
 [L51 C21] OPERATOR → "="
 [L51 C23] FLOATLIT → "2.2"
 [L51 C26] SEPARATOR → ","
 [L52 C9] KEYWORD → "double"
 [L52 C16] IDENTIFIER → "num3"
 [L52 C21] OPERATOR → "="
 [L52 C23] FLOATLIT → "3.3"
 [L52 C26] SEPARATOR → ","
 [L54 C9] IDENTIFIER → "String"
 [L54 C16] IDENTIFIER → "s1"
 [L54 C19] OPERATOR → "="
 [L54 C21] STRINGLIT → ""uno""
 [L54 C26] SEPARATOR → ","
 [L55 C9] IDENTIFIER → "String"
 [L55 C16] IDENTIFIER → "s2"
 [L55 C19] OPERATOR → "="
 [L55 C21] STRINGLIT → ""dos""
 [L55 C26] SEPARATOR → ","
 [L56 C9] IDENTIFIER → "String"
 [L56 C16] IDENTIFIER → "s3"
 [L56 C19] OPERATOR → "="

[L56 C21]	STRINGLIT	→ ""tres""
[L56 C27]	SEPARATOR	→ ","
[L59 C9]	KEYWORD	→ "if"
[L59 C12]	SEPARATOR	→ "("
[L59 C13]	IDENTIFIER	→ "num1"
[L59 C18]	OPERATOR	→ "<"
[L59 C20]	IDENTIFIER	→ "num2"
[L59 C25]	OPERATOR	→ " "
[L59 C28]	IDENTIFIER	→ "num3"
[L59 C33]	OPERATOR	→ ">"
[L59 C35]	IDENTIFIER	→ "num2"
[L59 C39]	SEPARATOR	→ ")"
[L59 C41]	SEPARATOR	→ "{"
[L60 C13]	IDENTIFIER	→ "System"
[L60 C19]	SEPARATOR	→ "."
[L60 C20]	IDENTIFIER	→ "out"
[L60 C23]	SEPARATOR	→ "."
[L60 C24]	IDENTIFIER	→ "println"
[L60 C31]	SEPARATOR	→ "("
[L60 C32]	STRINGLIT	→ ""Condición verdadera""
[L60 C53]	SEPARATOR	→ ")"
[L60 C54]	SEPARATOR	→ ","
[L61 C9]	SEPARATOR	→ "}"
[L64 C9]	KEYWORD	→ "for"
[L64 C13]	SEPARATOR	→ "("
[L64 C14]	KEYWORD	→ "int"
[L64 C18]	IDENTIFIER	→ "j"
[L64 C20]	OPERATOR	→ "="
[L64 C22]	INTEGERLIT	→ "0"
[L64 C23]	SEPARATOR	→ ","
[L64 C25]	IDENTIFIER	→ "j"
[L64 C27]	OPERATOR	→ "<"
[L64 C29]	INTEGERLIT	→ "5"
[L64 C30]	SEPARATOR	→ ","
[L64 C32]	IDENTIFIER	→ "j"
[L64 C33]	OPERATOR	→ "++"
[L64 C35]	SEPARATOR	→ ")"
[L64 C37]	SEPARATOR	→ "{"
[L65 C13]	KEYWORD	→ "for"
[L65 C17]	SEPARATOR	→ "("
[L65 C18]	KEYWORD	→ "int"
[L65 C22]	IDENTIFIER	→ "k"
[L65 C24]	OPERATOR	→ "="
[L65 C26]	INTEGERLIT	→ "0"
[L65 C27]	SEPARATOR	→ ","
[L65 C29]	IDENTIFIER	→ "k"
[L65 C31]	OPERATOR	→ "<"
[L65 C33]	INTEGERLIT	→ "3"

[L65 C34]	SEPARATOR	→ ";"
[L65 C36]	IDENTIFIER	→ "k"
[L65 C37]	OPERATOR	→ "++"
[L65 C39]	SEPARATOR	→ ")"
[L65 C41]	SEPARATOR	→ "{"
[L66 C17]	IDENTIFIER	→ "System"
[L66 C23]	SEPARATOR	→ "."
[L66 C24]	IDENTIFIER	→ "out"
[L66 C27]	SEPARATOR	→ "."
[L66 C28]	IDENTIFIER	→ "println"
[L66 C35]	SEPARATOR	→ "("
[L66 C36]	IDENTIFIER	→ "j"
[L66 C38]	OPERATOR	→ "+"
[L66 C40]	STRINGLIT	→ ""."
[L66 C44]	OPERATOR	→ "+"
[L66 C46]	IDENTIFIER	→ "k"
[L66 C47]	SEPARATOR	→ ")"
[L66 C48]	SEPARATOR	→ ";"
[L67 C13]	SEPARATOR	→ "}"
[L68 C9]	SEPARATOR	→ "}"
[L71 C9]	KEYWORD	→ "int"
[L71 C13]	IDENTIFIER	→ "x"
[L71 C15]	OPERATOR	→ "="
[L71 C17]	INTEGERLIT	→ "5"
[L71 C18]	SEPARATOR	→ ";"
[L72 C9]	IDENTIFIER	→ "x"
[L72 C11]	OPERATOR	→ "+="
[L72 C14]	INTEGERLIT	→ "2"
[L72 C15]	SEPARATOR	→ ";"
[L73 C9]	IDENTIFIER	→ "x"
[L73 C11]	OPERATOR	→ "-="
[L73 C14]	INTEGERLIT	→ "1"
[L73 C15]	SEPARATOR	→ ";"
[L74 C9]	IDENTIFIER	→ "x"
[L74 C11]	OPERATOR	→ "*="
[L74 C14]	INTEGERLIT	→ "3"
[L74 C15]	SEPARATOR	→ ";"
[L75 C9]	IDENTIFIER	→ "x"
[L75 C11]	OPERATOR	→ "/="
[L75 C14]	INTEGERLIT	→ "2"
[L75 C15]	SEPARATOR	→ ";"
[L76 C9]	IDENTIFIER	→ "x"
[L76 C11]	OPERATOR	→ "%="
[L76 C14]	INTEGERLIT	→ "2"
[L76 C15]	SEPARATOR	→ ";"
[L78 C9]	KEYWORD	→ "boolean"
[L78 C17]	IDENTIFIER	→ "flag"
[L78 C22]	OPERATOR	→ "="

[L78 C24]	KEYWORD	→ "true"
[L78 C28]	SEPARATOR	→ ";
[L79 C9]	KEYWORD	→ "if"
[L79 C12]	SEPARATOR	→ "("
[L79 C13]	OPERATOR	→ "!"
[L79 C14]	IDENTIFIER	→ "flag"
[L79 C18]	SEPARATOR	→ ")"
[L79 C20]	SEPARATOR	→ "{"
[L80 C13]	IDENTIFIER	→ "System"
[L80 C19]	SEPARATOR	→ ".
[L80 C20]	IDENTIFIER	→ "out"
[L80 C23]	SEPARATOR	→ ".
[L80 C24]	IDENTIFIER	→ "println"
[L80 C31]	SEPARATOR	→ "("
[L80 C32]	STRINGLIT	→ ""Falso""
[L80 C39]	SEPARATOR	→ ")"
[L80 C40]	SEPARATOR	→ ";
[L81 C9]	SEPARATOR	→ "}"
[L84 C9]	KEYWORD	→ "int"
[L84 C13]	IDENTIFIER	→ "opcion"
[L84 C20]	OPERATOR	→ "="
[L84 C22]	INTEGERLIT	→ "2"
[L84 C23]	SEPARATOR	→ ";
[L85 C9]	KEYWORD	→ "switch"
[L85 C16]	SEPARATOR	→ "("
[L85 C17]	IDENTIFIER	→ "opcion"
[L85 C23]	SEPARATOR	→ ")"
[L85 C25]	SEPARATOR	→ "{"
[L86 C13]	KEYWORD	→ "case"
[L86 C18]	INTEGERLIT	→ "1"
[L86 C19]	OPERATOR	→ ":
[L87 C17]	IDENTIFIER	→ "System"
[L87 C23]	SEPARATOR	→ ".
[L87 C24]	IDENTIFIER	→ "out"
[L87 C27]	SEPARATOR	→ ".
[L87 C28]	IDENTIFIER	→ "println"
[L87 C35]	SEPARATOR	→ "("
[L87 C36]	STRINGLIT	→ ""Uno""
[L87 C41]	SEPARATOR	→ ")"
[L87 C42]	SEPARATOR	→ ";
[L88 C17]	KEYWORD	→ "break"
[L88 C22]	SEPARATOR	→ ";
[L89 C13]	KEYWORD	→ "case"
[L89 C18]	INTEGERLIT	→ "2"
[L89 C19]	OPERATOR	→ ":
[L90 C17]	IDENTIFIER	→ "System"
[L90 C23]	SEPARATOR	→ ".
[L90 C24]	IDENTIFIER	→ "out"

[L90 C27]	SEPARATOR	→ "."
[L90 C28]	IDENTIFIER	→ "println"
[L90 C35]	SEPARATOR	→ "("
[L90 C36]	STRINGLIT	→ ""Dos""
[L90 C41]	SEPARATOR	→ ")"
[L90 C42]	SEPARATOR	→ ";"
[L91 C17]	KEYWORD	→ "break"
[L91 C22]	SEPARATOR	→ ";"
[L92 C13]	KEYWORD	→ "default"
[L92 C20]	OPERATOR	→ ":"
[L93 C17]	IDENTIFIER	→ "System"
[L93 C23]	SEPARATOR	→ "."
[L93 C24]	IDENTIFIER	→ "out"
[L93 C27]	SEPARATOR	→ "."
[L93 C28]	IDENTIFIER	→ "println"
[L93 C35]	SEPARATOR	→ "("
[L93 C36]	STRINGLIT	→ ""Otro""
[L93 C42]	SEPARATOR	→ ")"
[L93 C43]	SEPARATOR	→ ";"
[L94 C9]	SEPARATOR	→ "}"
[L97 C9]	KEYWORD	→ "int"
[L97 C13]	IDENTIFIER	→ "v1"
[L97 C16]	OPERATOR	→ "="
[L97 C18]	INTEGERLIT	→ "1"
[L97 C19]	SEPARATOR	→ ";"
[L98 C9]	KEYWORD	→ "int"
[L98 C13]	IDENTIFIER	→ "v2"
[L98 C16]	OPERATOR	→ "="
[L98 C18]	INTEGERLIT	→ "2"
[L98 C19]	SEPARATOR	→ ";"
[L99 C9]	KEYWORD	→ "int"
[L99 C13]	IDENTIFIER	→ "v3"
[L99 C16]	OPERATOR	→ "="
[L99 C18]	INTEGERLIT	→ "3"
[L99 C19]	SEPARATOR	→ ";"
[L100 C9]	KEYWORD	→ "int"
[L100 C13]	IDENTIFIER	→ "v4"
[L100 C16]	OPERATOR	→ "="
[L100 C18]	INTEGERLIT	→ "4"
[L100 C19]	SEPARATOR	→ ";"
[L101 C9]	KEYWORD	→ "int"
[L101 C13]	IDENTIFIER	→ "v5"
[L101 C16]	OPERATOR	→ "="
[L101 C18]	INTEGERLIT	→ "5"
[L101 C19]	SEPARATOR	→ ";"
[L102 C9]	KEYWORD	→ "int"
[L102 C13]	IDENTIFIER	→ "v6"
[L102 C16]	OPERATOR	→ "="

[L102 C18]	INTEGERLIT	→ "6"
[L102 C19]	SEPARATOR	→ ";
[L103 C9]	KEYWORD	→ "int"
[L103 C13]	IDENTIFIER	→ "v7"
[L103 C16]	OPERATOR	→ "="
[L103 C18]	INTEGERLIT	→ "7"
[L103 C19]	SEPARATOR	→ ";
[L104 C9]	KEYWORD	→ "int"
[L104 C13]	IDENTIFIER	→ "v8"
[L104 C16]	OPERATOR	→ "="
[L104 C18]	INTEGERLIT	→ "8"
[L104 C19]	SEPARATOR	→ ";
[L105 C9]	KEYWORD	→ "int"
[L105 C13]	IDENTIFIER	→ "v9"
[L105 C16]	OPERATOR	→ "="
[L105 C18]	INTEGERLIT	→ "9"
[L105 C19]	SEPARATOR	→ ";
[L106 C9]	KEYWORD	→ "int"
[L106 C13]	IDENTIFIER	→ "v10"
[L106 C17]	OPERATOR	→ "="
[L106 C19]	INTEGERLIT	→ "10"
[L106 C21]	SEPARATOR	→ ";
[L109 C9]	IDENTIFIER	→ "String"
[L109 C16]	IDENTIFIER	→ "t1"
[L109 C19]	OPERATOR	→ "="
[L109 C21]	STRINGLIT	→ ""a""
[L109 C24]	SEPARATOR	→ ";
[L110 C9]	IDENTIFIER	→ "String"
[L110 C16]	IDENTIFIER	→ "t2"
[L110 C19]	OPERATOR	→ "="
[L110 C21]	STRINGLIT	→ ""b""
[L110 C24]	SEPARATOR	→ ";
[L111 C9]	IDENTIFIER	→ "String"
[L111 C16]	IDENTIFIER	→ "t3"
[L111 C19]	OPERATOR	→ "="
[L111 C21]	STRINGLIT	→ ""c""
[L111 C24]	SEPARATOR	→ ";
[L112 C9]	IDENTIFIER	→ "String"
[L112 C16]	IDENTIFIER	→ "t4"
[L112 C19]	OPERATOR	→ "="
[L112 C21]	STRINGLIT	→ ""d""
[L112 C24]	SEPARATOR	→ ";
[L113 C9]	IDENTIFIER	→ "String"
[L113 C16]	IDENTIFIER	→ "t5"
[L113 C19]	OPERATOR	→ "="
[L113 C21]	STRINGLIT	→ ""e""
[L113 C24]	SEPARATOR	→ ";
[L116 C9]	KEYWORD	→ "if"

[L116 C12]	SEPARATOR	→ "("
[L116 C13]	IDENTIFIER	→ "v1"
[L116 C16]	OPERATOR	→ "=="
[L116 C19]	INTEGERLIT	→ "1"
[L116 C21]	OPERATOR	→ "&&"
[L116 C24]	IDENTIFIER	→ "v2"
[L116 C27]	OPERATOR	→ "=="
[L116 C30]	INTEGERLIT	→ "2"
[L116 C31]	SEPARATOR	→ ")"
[L116 C33]	SEPARATOR	→ "{"
[L117 C13]	IDENTIFIER	→ "System"
[L117 C19]	SEPARATOR	→ "."
[L117 C20]	IDENTIFIER	→ "out"
[L117 C23]	SEPARATOR	→ "."
[L117 C24]	IDENTIFIER	→ "println"
[L117 C31]	SEPARATOR	→ "("
[L117 C32]	STRINGLIT	→ ""Correcto""
[L117 C42]	SEPARATOR	→ ")"
[L117 C43]	SEPARATOR	→ ";
[L118 C9]	SEPARATOR	→ "}"
[L120 C9]	KEYWORD	→ "if"
[L120 C12]	SEPARATOR	→ "("
[L120 C13]	IDENTIFIER	→ "v3"
[L120 C16]	OPERATOR	→ "!="
[L120 C19]	INTEGERLIT	→ "0"
[L120 C20]	SEPARATOR	→ ")"
[L120 C22]	SEPARATOR	→ "{"
[L121 C13]	IDENTIFIER	→ "System"
[L121 C19]	SEPARATOR	→ "."
[L121 C20]	IDENTIFIER	→ "out"
[L121 C23]	SEPARATOR	→ "."
[L121 C24]	IDENTIFIER	→ "println"
[L121 C31]	SEPARATOR	→ "("
[L121 C32]	STRINGLIT	→ ""No es cero""
[L121 C44]	SEPARATOR	→ ")"
[L121 C45]	SEPARATOR	→ ";
[L122 C9]	SEPARATOR	→ "}"

Preguntas

1. ¿Qué pasa si eliminas los \b del patrón KEYWORD?

Los \b aseguran que se reconozcan palabras completas.

Si se eliminan, el lexer puede detectar keywords dentro de identificadores.
Por ejemplo, en print podría detectar int, lo cual es incorrecto.

2. ¿Por qué el patrón KEYWORD debe evaluarse antes que IDENTIFIER?

Porque las palabras clave también cumplen el patrón de identificadores.

Si IDENTIFIER se evalúa primero, palabras como if o while serían clasificadas como identificadores en lugar de keywords.

El orden evita esta ambigüedad.

3. ¿Podrías construir una sola ER para ambos tokens?

Sí, se puede usar una sola expresión regular para identificadores:

```
[a-zA-Z_$][a-zA-Z0-9_$]*
```

Y después verificar en código si el lexema es una keyword.

Sin embargo, no es recomendable porque mezcla lógica léxica con lógica semántica.

4. ¿Cómo manejarías el operador ternario ?:?

El lexer lo separa en dos tokens:

?

:

No se manejan como uno solo.

El parser es quien interpreta que juntos forman un operador ternario.

5. ¿Por qué >>> debe reconocerse antes que >>?

Porque el lexer toma la primera coincidencia válida.

Si >> está antes, entonces >>> se dividiría en:

>>

>

Esto sería incorrecto.

Por eso los operadores más largos deben ir primero.

6. ¿Qué diferencia hay entre un separador y un operador?

Separadores: delimitan estructura (ej. { } () ; ,)

Operadores: realizan operaciones (ej. +, ==, &&)

En la gramática, los separadores organizan el código y los operadores forman expresiones.

7. ¿Por qué 08 no es un octal válido en Java?

Porque los números octales solo permiten dígitos del 0 al 7.

El 8 no es válido en base 8, por lo que 08 es incorrecto.

El regex se ajusta restringiendo a [0-7].

8. ¿Cómo extenderías el patrón STRING para soportar Text Blocks (""" """)?

Se agrega un nuevo patrón como:

```
\"\"\"[\\s\\S]*?\"\"\"
```

Este permite reconocer cadenas multilínea con triple comillas, introducidas en Java 15.

9. ¿Tiene sentido que el lexer verifique si un número está en rango?

No.

El lexer solo reconoce la forma del número (estructura).

La validación de rango corresponde al análisis semántico o al parser.

10. ¿Cómo manejaría el lexer un comentario de bloque no cerrado?

Si no encuentra */, el patrón no hace match.

Esto se detecta como error léxico, ya que queda texto sin consumir.

El lexer debe reportarlo y continuar.

11. ¿Deberían los JavaDoc comments producir un token diferente?

Sí.

Porque los JavaDoc (/** */) tienen significado especial para documentación.

Pueden ser útiles en análisis posterior, a diferencia de los comentarios normales.

12. ¿Cuál es la diferencia entre recuperación de errores en el lexer vs parser?

Lexer:

Detecta errores simples (caracteres inválidos)

Se recupera avanzando al siguiente carácter

Parser:

Detecta errores de estructura (sintaxis)

Usa reglas más complejas para continuar